Graphics in R

Matteo Fasiolo

Graphics in R

R has powerful graphics features, which allows users to visualize data easily.

You can type `demo("graphics")` in the command line to see demo code/plots.

You may have already seen some of the graphics functions, so we can be brief.

NOTE: we will cover R plotting from the `graphics` package.

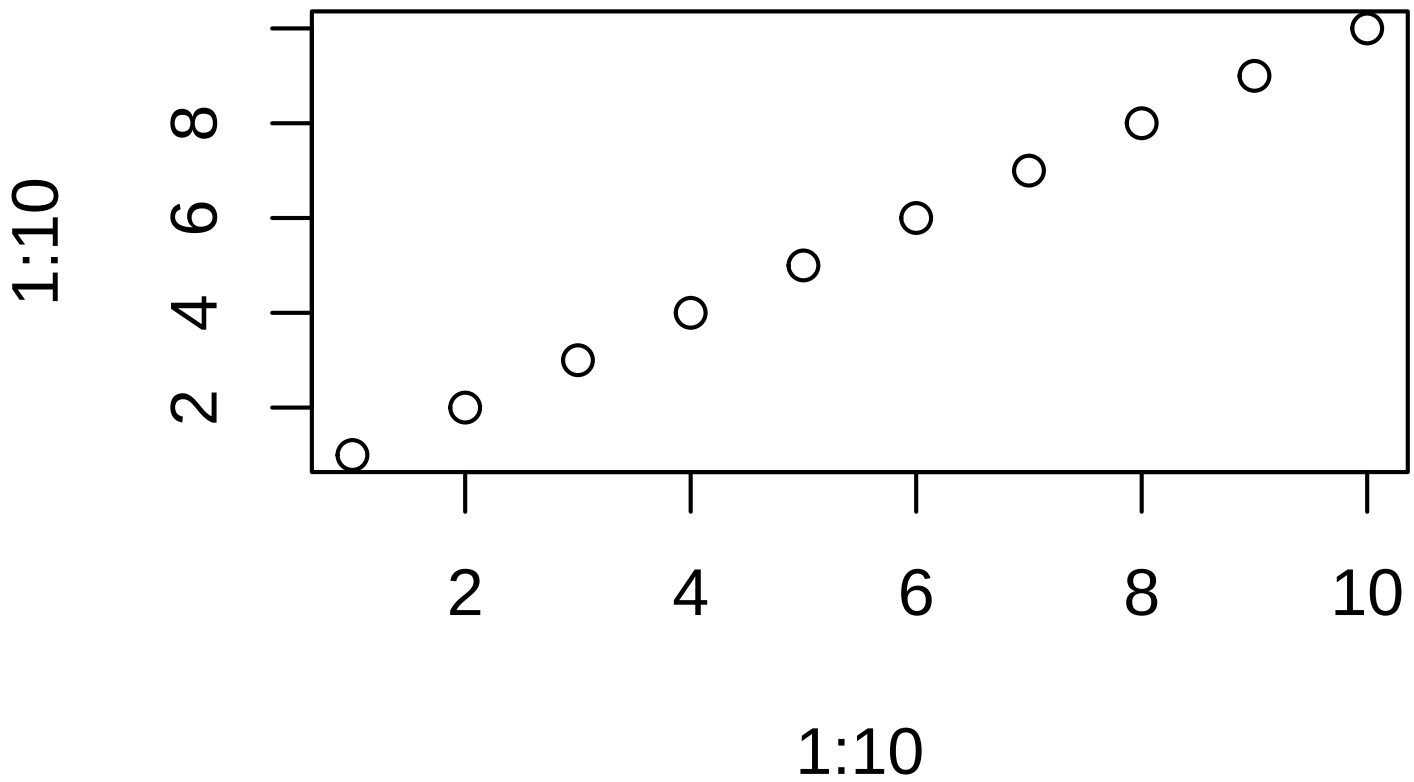
We will not cover plotting with `ggplot2` package, e.g.:

```
ggplot(data, aes(x = x, y = y)) + geom_line()
```

Please **do NOT use it** in coursework, problem sheets etc.

Create a plot: `plot`

```
plot(x = 1:10, y = 1:10)
```

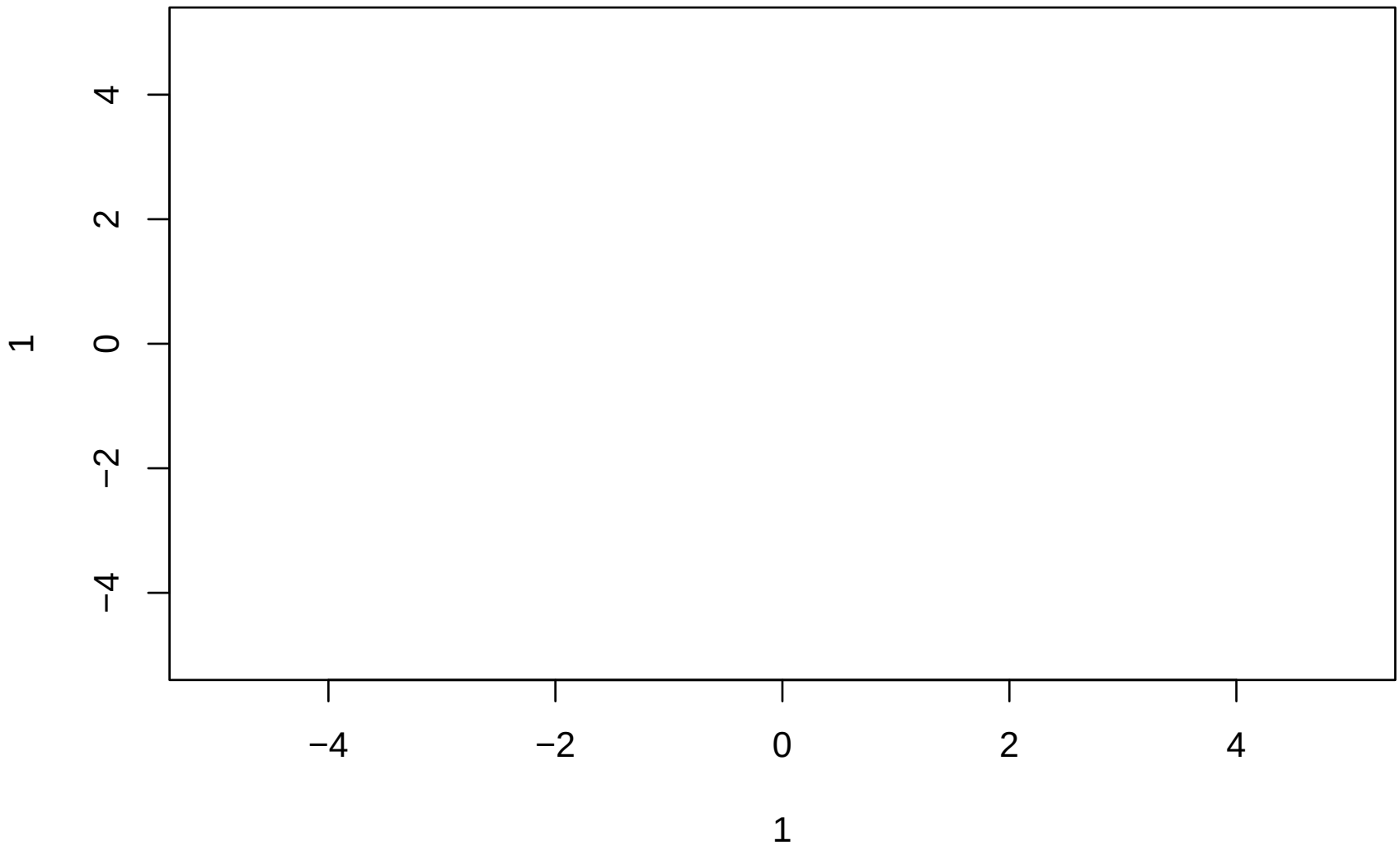


We are calling `plot()` for its **side effects**:

```
a <- plot(x = 1:10, y = 1:10)
a
NULL
```

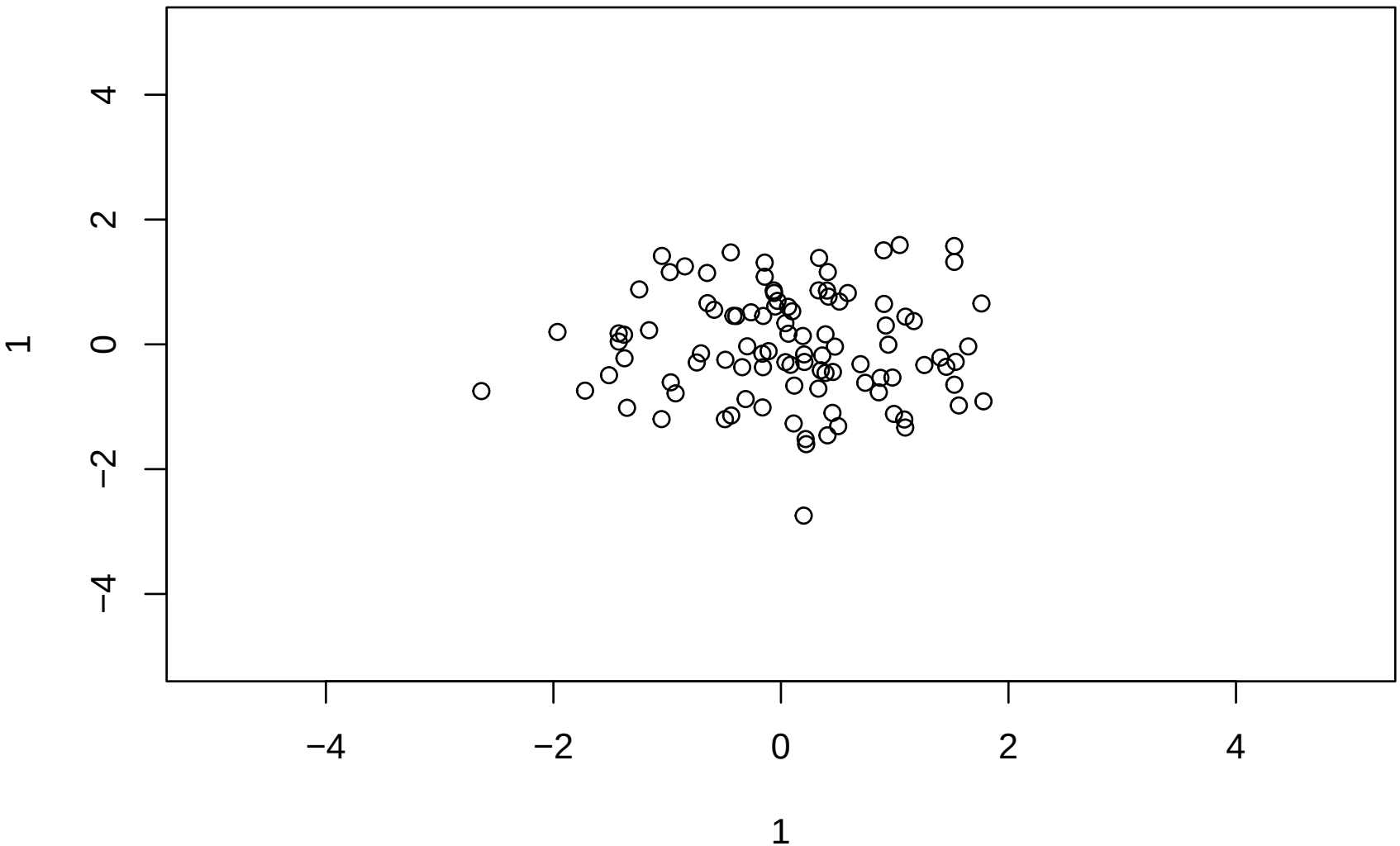
We can use it to produce an empty plot:

```
plot(x = 1, y = 1, type = "n",
     xlim = c(-5, 5), ylim = c(-5, 5))
```



and then fill it with, e.g., points:

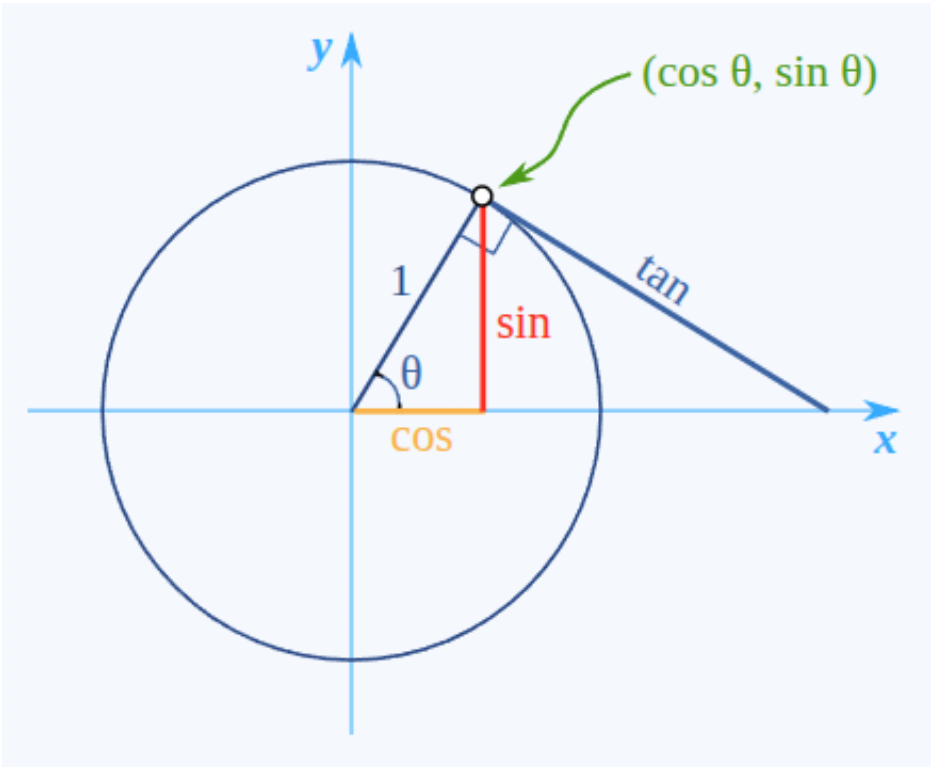
```
x <- rnorm(100)
y <- rnorm(100)
points(x,y)
```



Suppose we want to draw a circle.

What are its x and y coordinates for `plot(x = ?, y = ?)`

We can plot $x = \cos(\theta)$ vs $y = \sin(\theta)$ for $\theta \in (0, 2\pi)$.



From https://www.mathsisfun.com/geometry/unit-circle.html

```r
theta <- seq(0, 2 * pi, length.out = 50)
```

```r
x_c <- cos(theta)
y_c <- sin(theta)
```

First attempt:

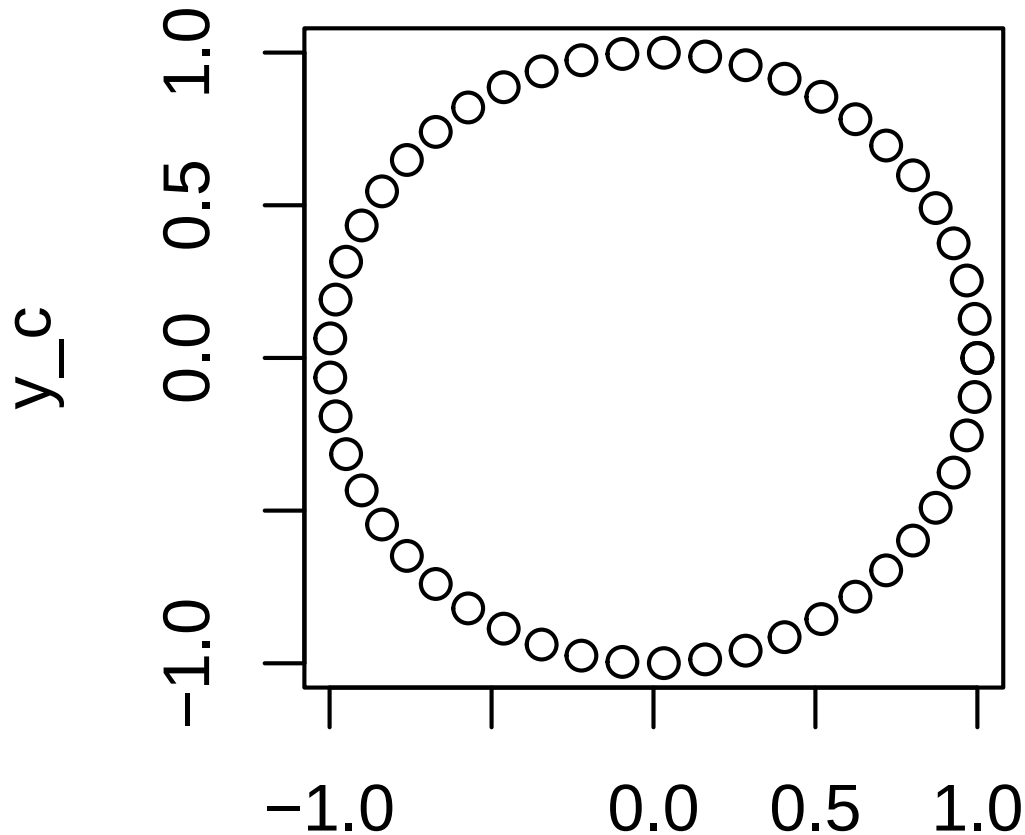
```r
plot(x_c, y_c)
```

```r
theta <- seq(0, 2 * pi, length.out = 50)
```

```r
x_c <- cos(theta)
y_c <- sin(theta)
```

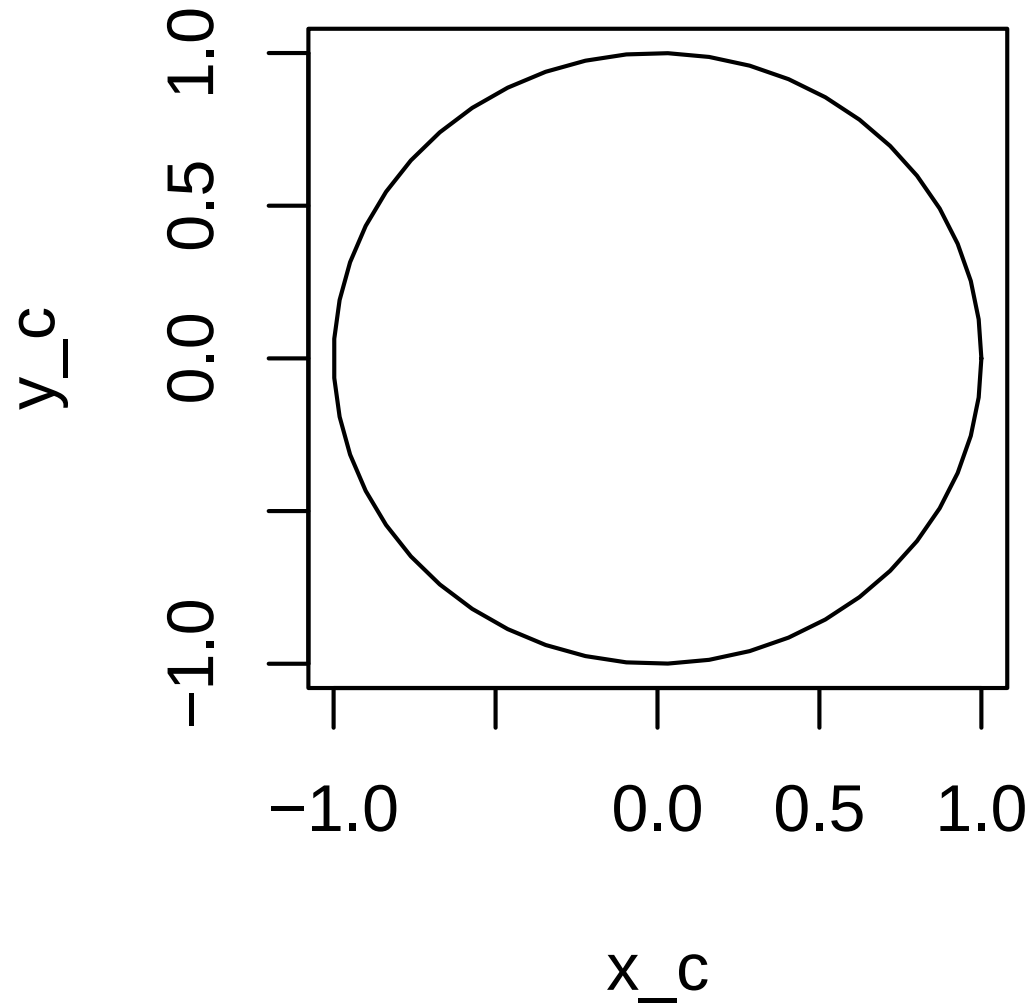
First attempt:

```r
plot(x_c, y_c)
```



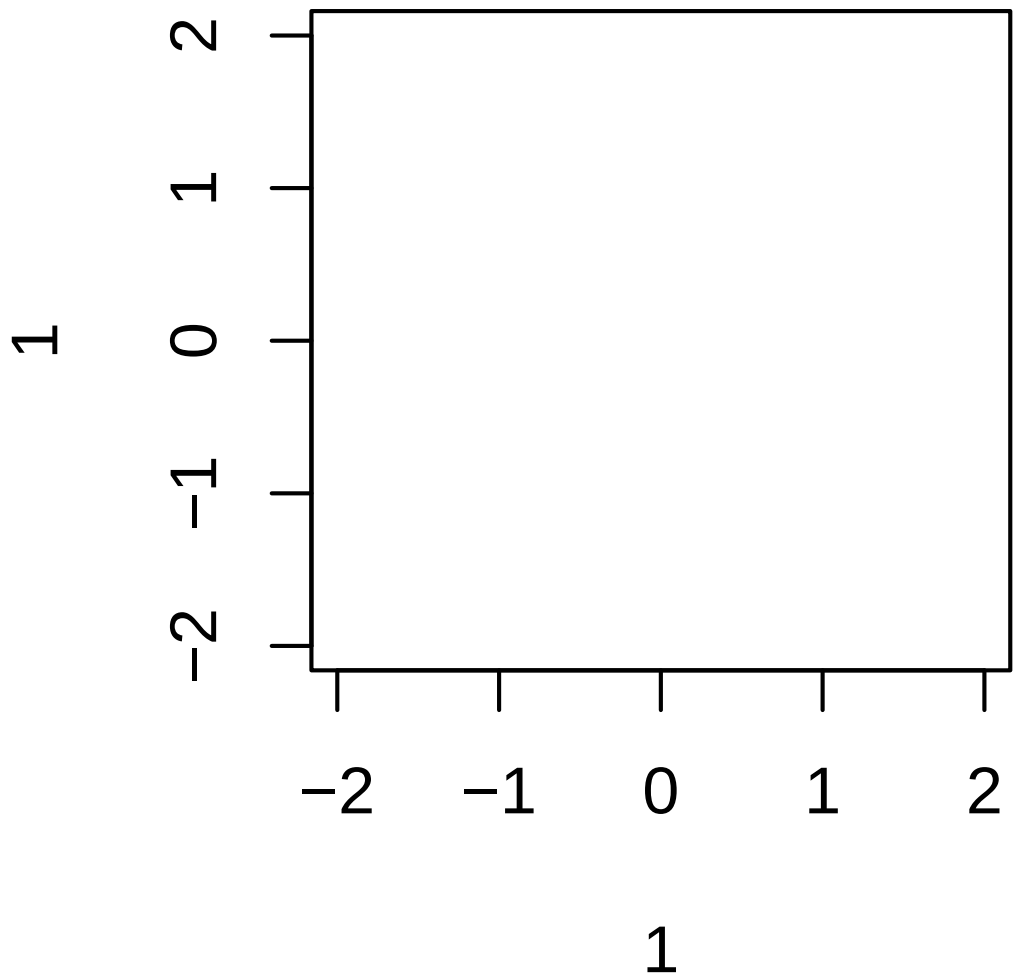
Better:

```
plot(x = x_c, y = y_c, type = "l")
```



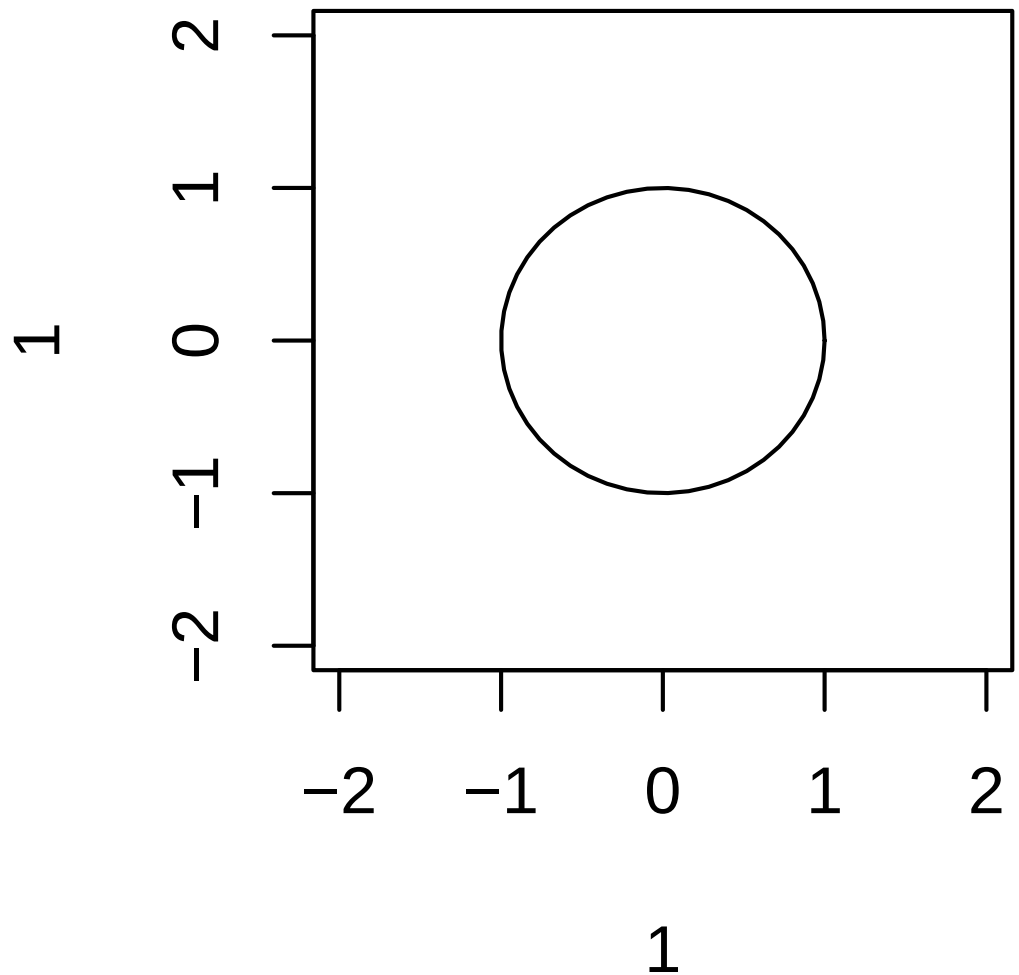
Or in two steps, using `lines`:

```
plot(1, 1, type = "n",
     xlim = c(-2, 2), ylim = c(-2, 2))
```



Or in two steps, using `lines`:

```
plot(1, 1, type = "n",
     xlim = c(-2, 2), ylim = c(-2, 2))
lines(x_c, y_c)
```

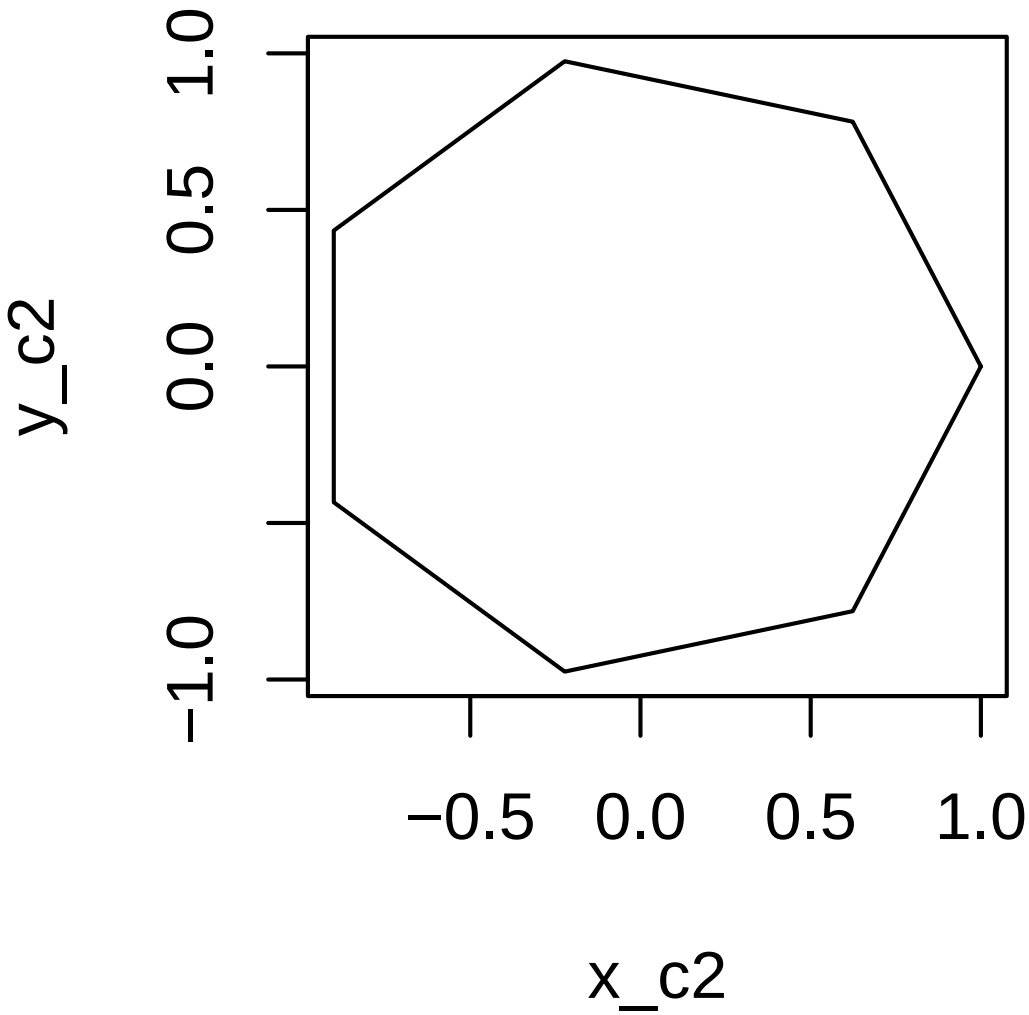


Note that R is really drawing a polygon:

```
theta2 <- seq(0, 2*pi, length.out = 8)
x_c2 <- cos(theta2)
y_c2 <- sin(theta2)
plot(x_c2, y_c2, type = "l")
```

Note that R is really drawing a polygon:

```
theta2 <- seq(0, 2*pi, length.out = 8)
x_c2 <- cos(theta2)
y_c2 <- sin(theta2)
plot(x_c2, y_c2, type = "l")
```



Note that the order matters:

```
square <- rbind(c(0, 0, 1, 1, 0),
                c(0, 1, 1, 0, 0))
square
```

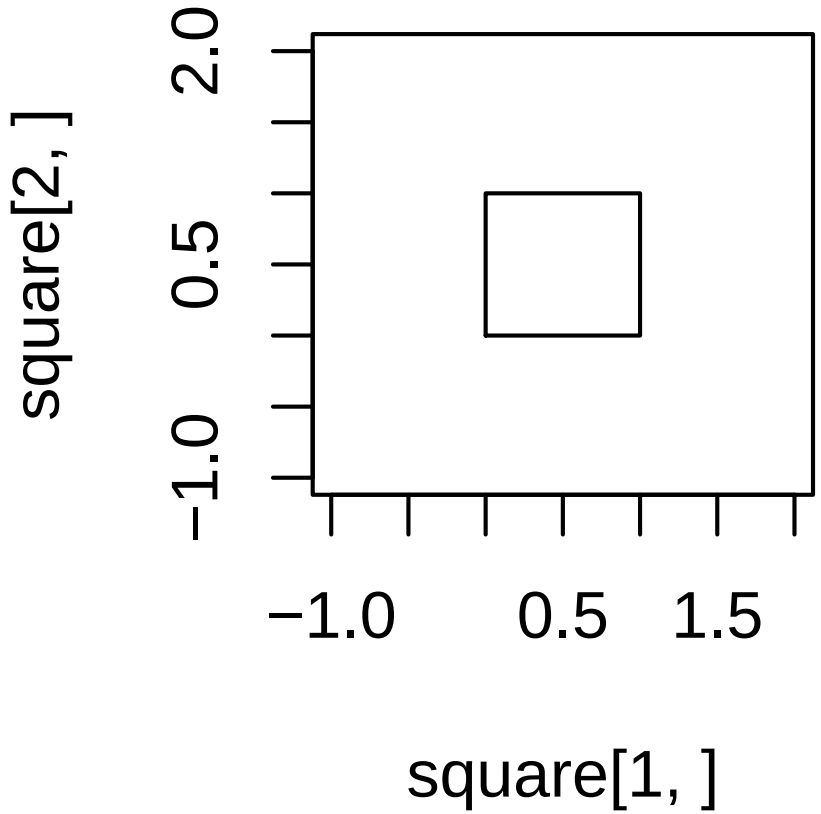
```
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    0    1    1    0
[2,]    0    1    1    0    0
```

```
plot(square[1, ], square[2, ], type = "l",
     ylim = c(-1, 2), xlim = c(-1, 2))
```

Note that the order matters:

```
square <- rbind(c(0, 0, 1, 1, 0),
                c(0, 1, 1, 0, 0))
```

```
plot(square[1, ], square[2, ], type = "l",
     ylim = c(-1, 2), xlim = c(-1, 2))
```



```
square
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    0    1    1    0
[2,]    0    1    1    0    0
```

```
square <- square[ , c(1, 3, 2, 4, 5)]
```

```
square
```

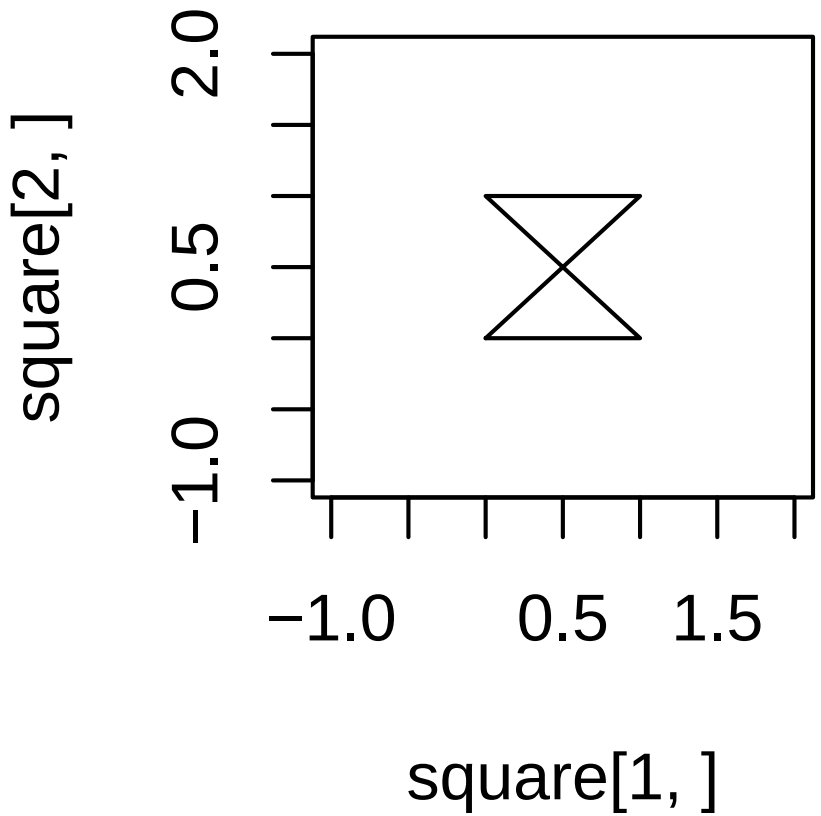
```
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    1    0    1    0
[2,]    0    1    1    0    0
```

```
plot(square[1, ], square[2, ], type = "l",
     ylim = c(-1, 2), xlim = c(-1, 2))
```

```
square
```

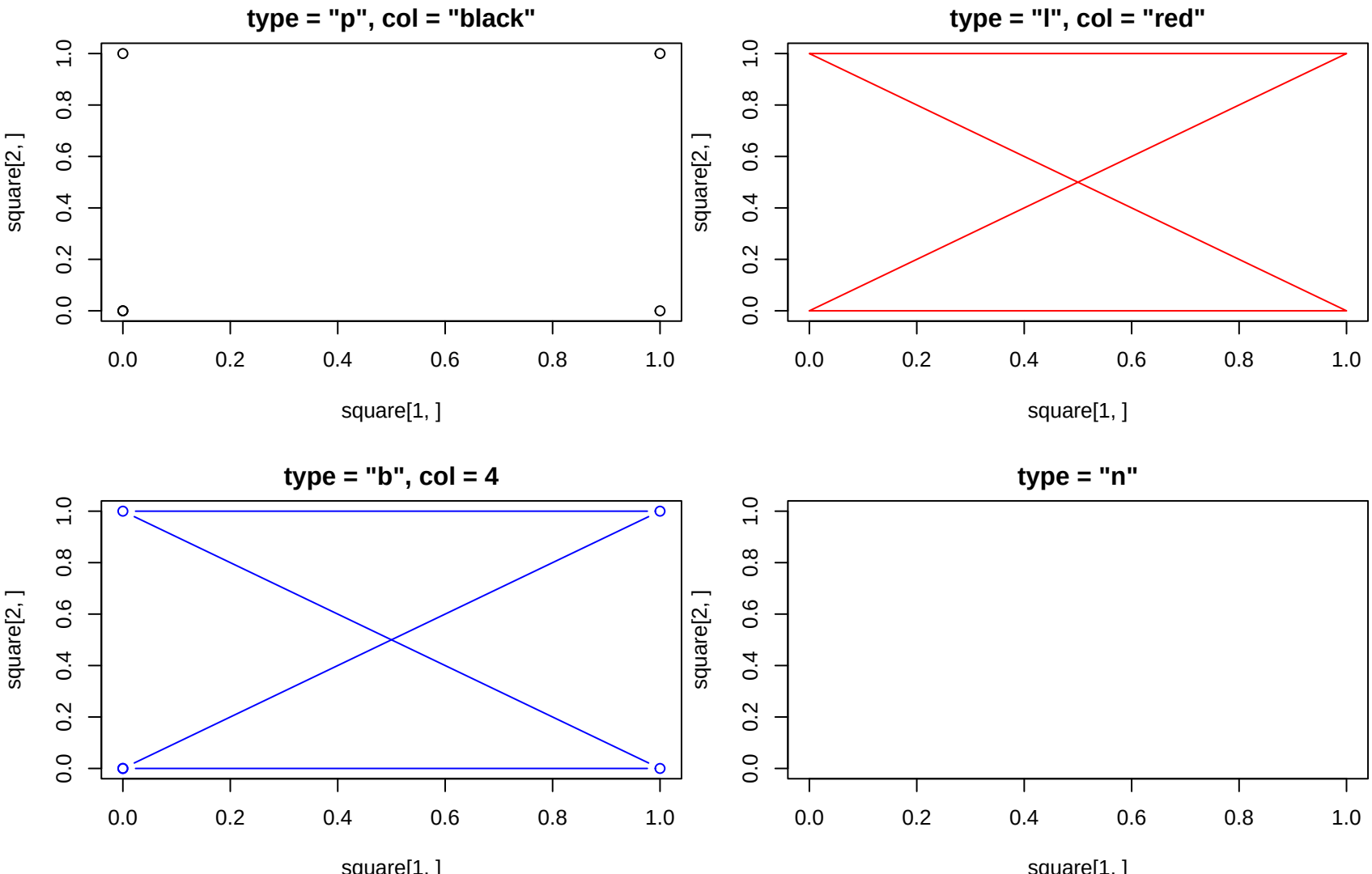
```
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    1    0    1    0
[2,]    0    1    1    0    0
```

```
plot(square[1, ], square[2, ], type = "l",
     ylim = c(-1, 2), xlim = c(-1, 2))
```



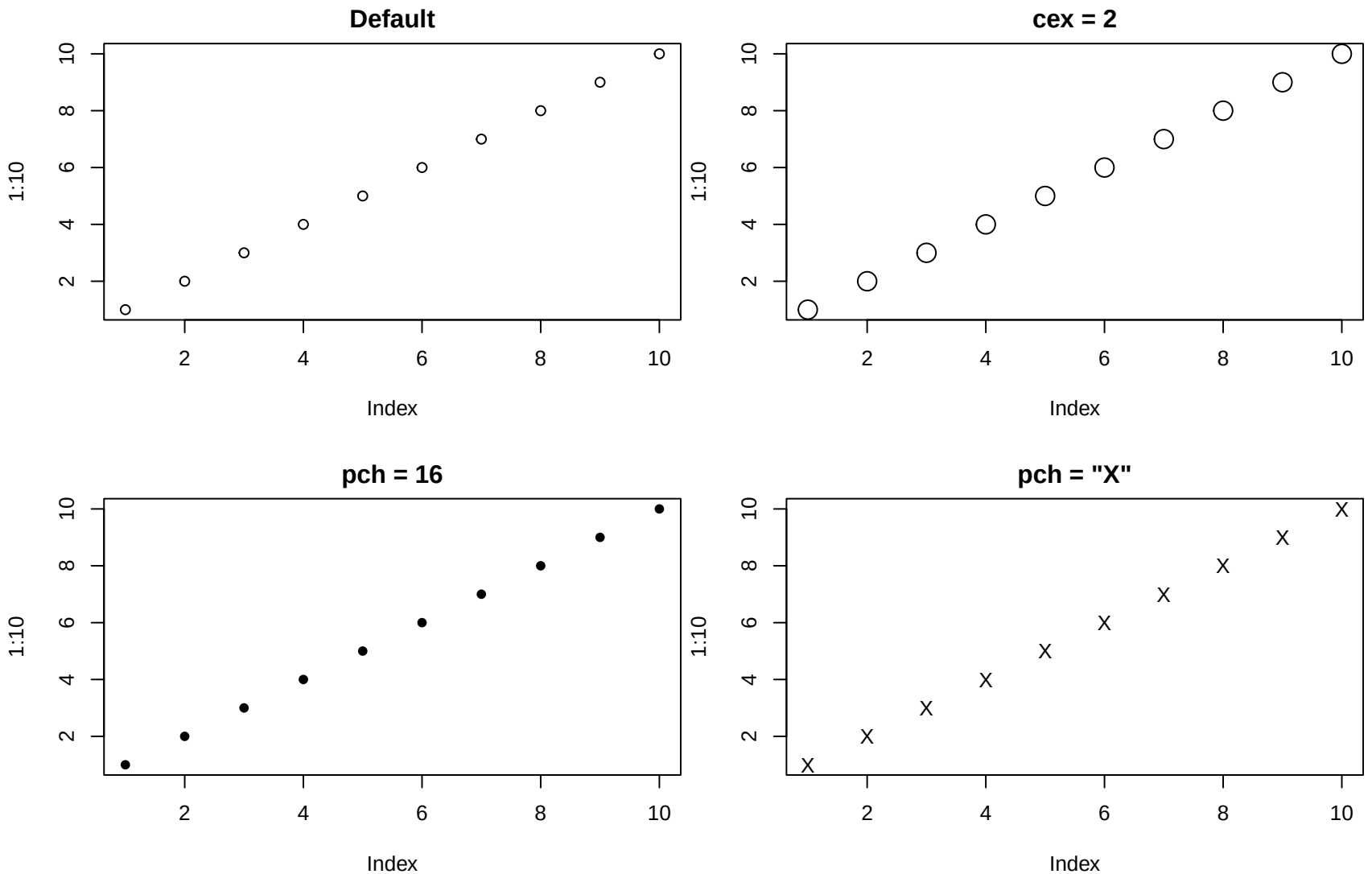
Common aesthetics

- `type = "p"` or “l” or “b” or “n”: points, lines, both, none.
- `col`: color



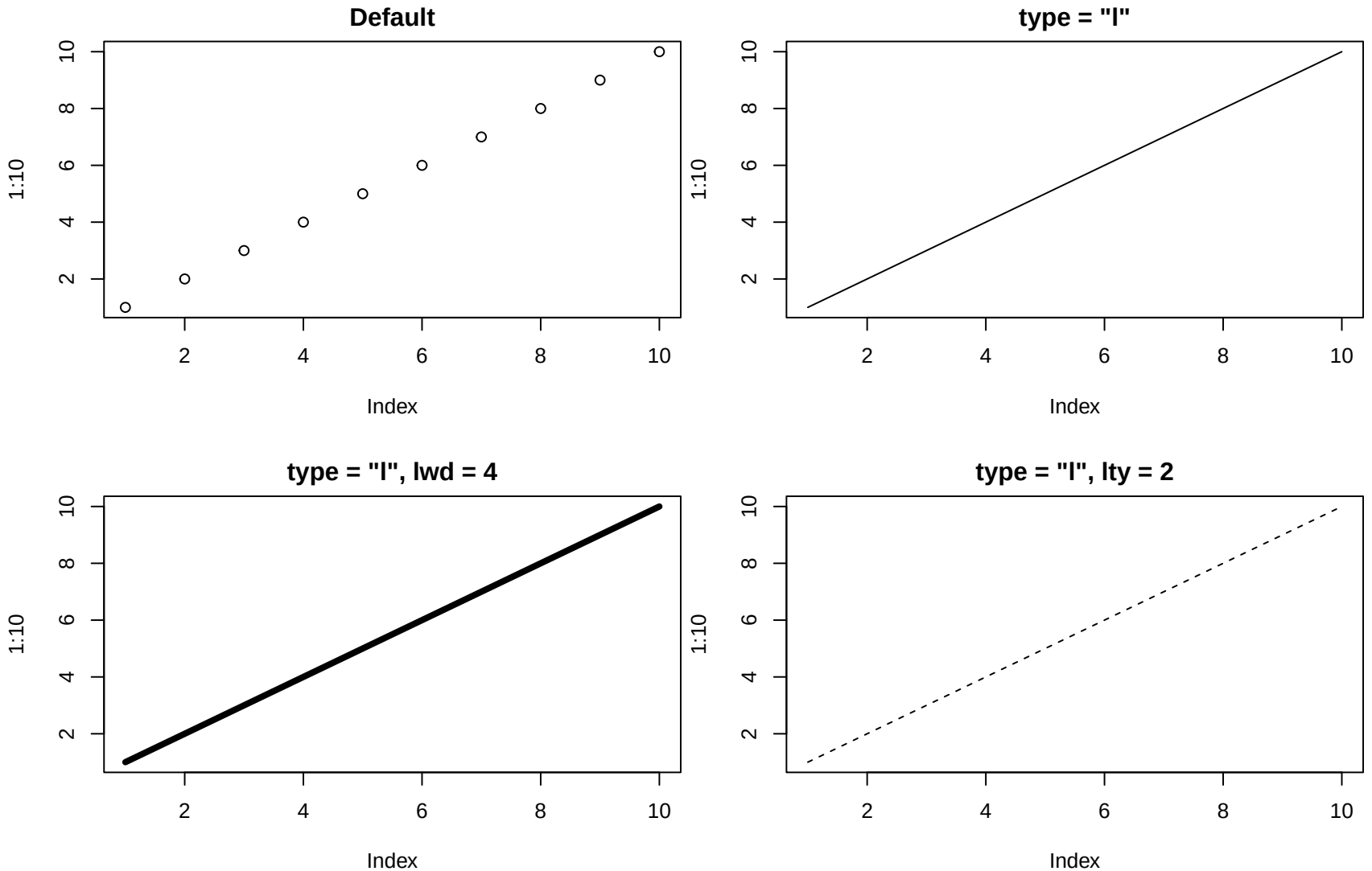
- `cex`: point/label expansion (size)
- `pch`: point symbol (e.g., 1–25, or characters like `"+"`, `"x"`)

```
plot(1:10, cex = ???, pch = ???)
```



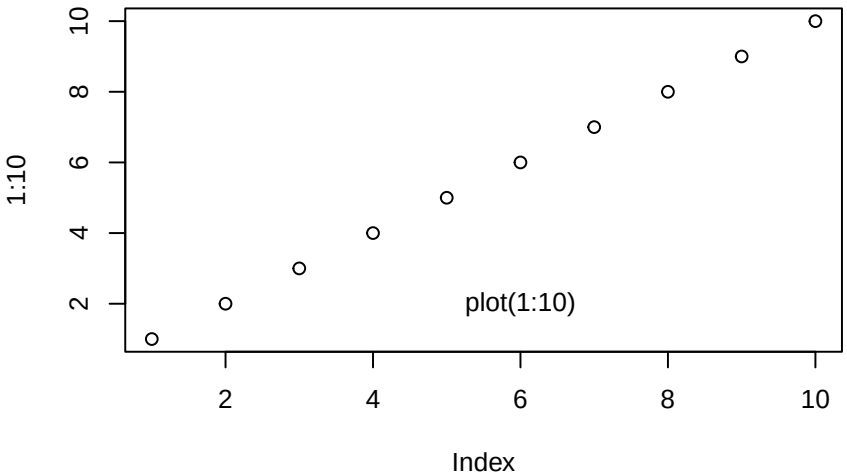
`lty`, `lwd`: line type (1–6) and width

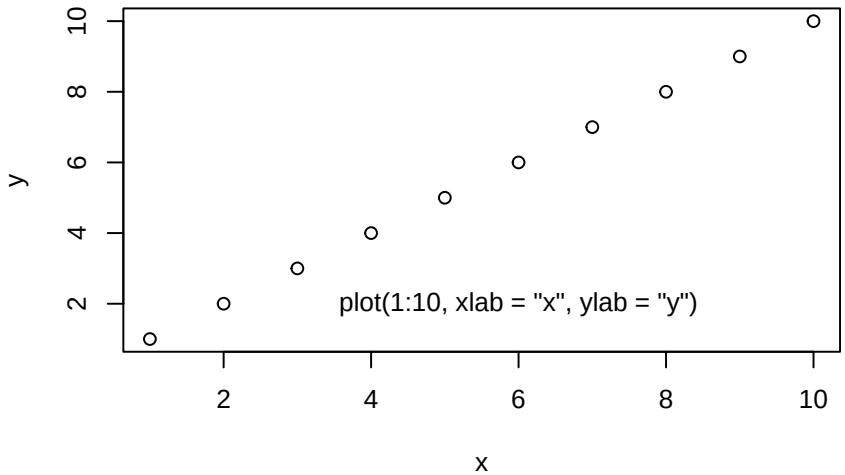
```
plot(1:10, type = ???, lty = ???, lwd = ???)
```

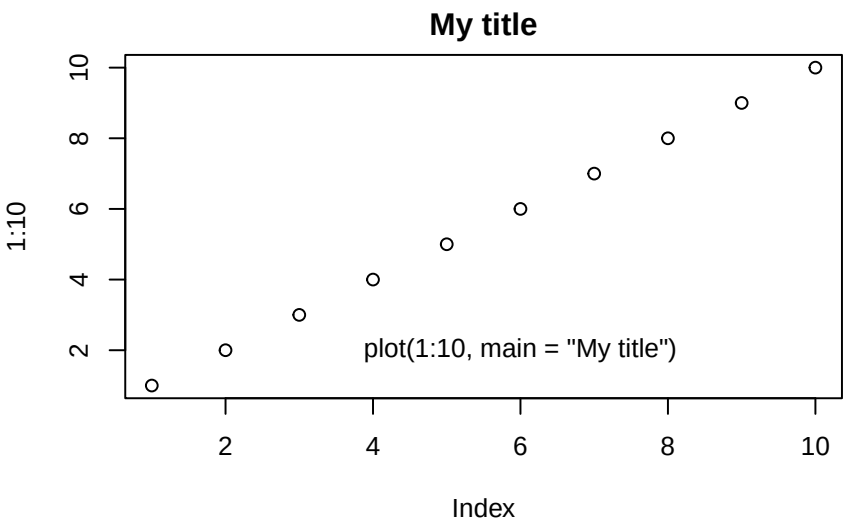


`xlab`, `ylab`, `main`: axis labels, main title

```
plot(1:10, xlab = ???, ylab = ???, main = ???)
```



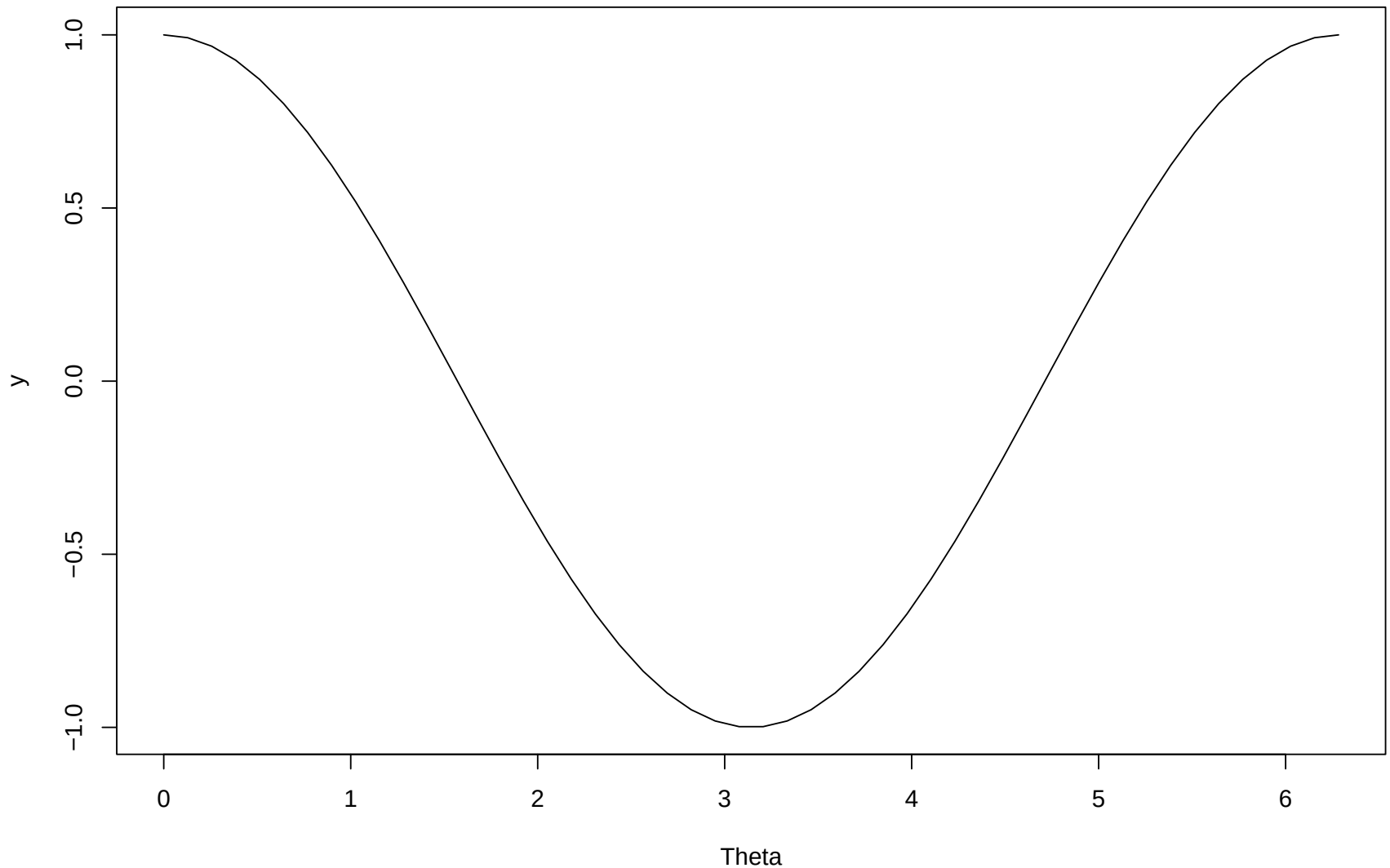




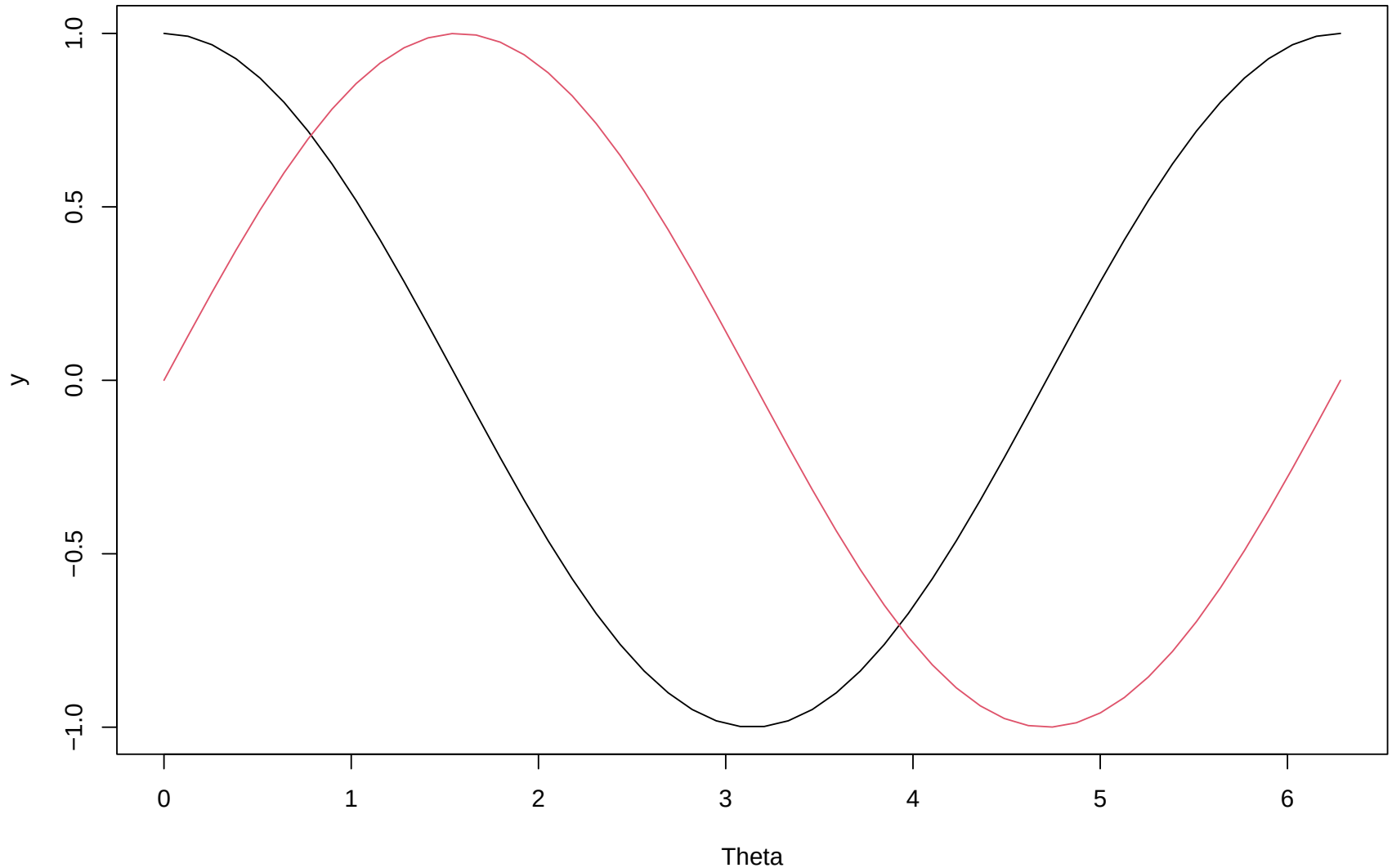
R graphics are layered

You can add graphical layers to a plot:

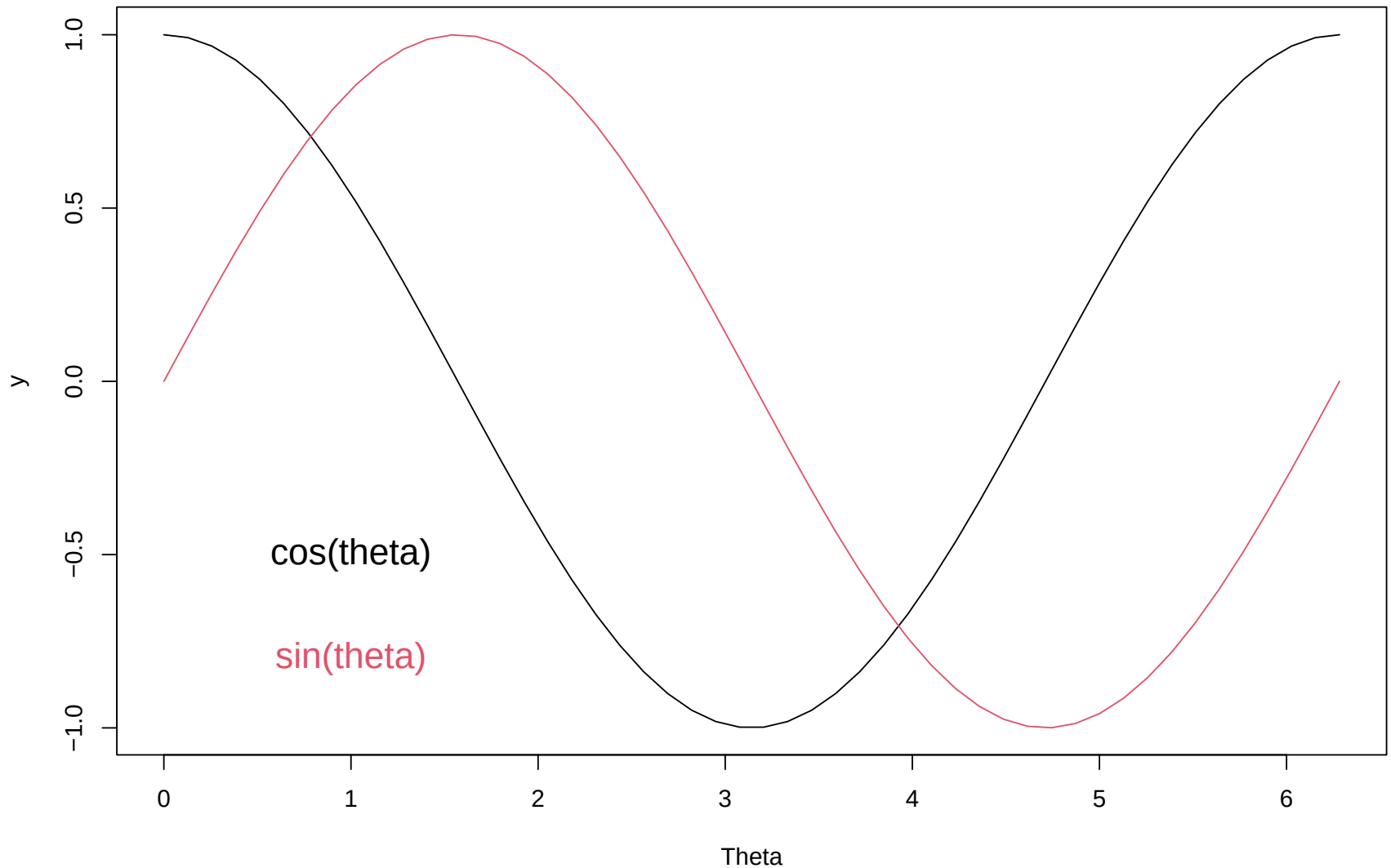
```
plot(theta, x_c, type = "l", xlab = "Theta", ylab = "y")
```



```r
plot(theta, x_c, type = "l", xlab = "Theta", ylab = "y")
lines(theta, y_c, col = 2)
```



```r
plot(theta, x_c, type = "l", xlab = "Theta", ylab = "y")
lines(theta, y_c, col = 2)
text(x = 1, y = -0.5, "cos(theta)", cex = 1.5)
text(x = 1, y = -0.8, "sin(theta)", cex = 1.5, col = 2)
```



Getting and setting global graphical parameters

The way R renders your plot on the screen depends on the global graphical parameters.

You can get them via:

```
par()
```

```
$xlog
[1] FALSE

$ylog
[1] FALSE

$adj
[1] 0.5

$ann
[1] TRUE
```

There are **a lot** of them:

```
length( par() )
```

```
[1] 72
```

For example:

```
par()$col
```

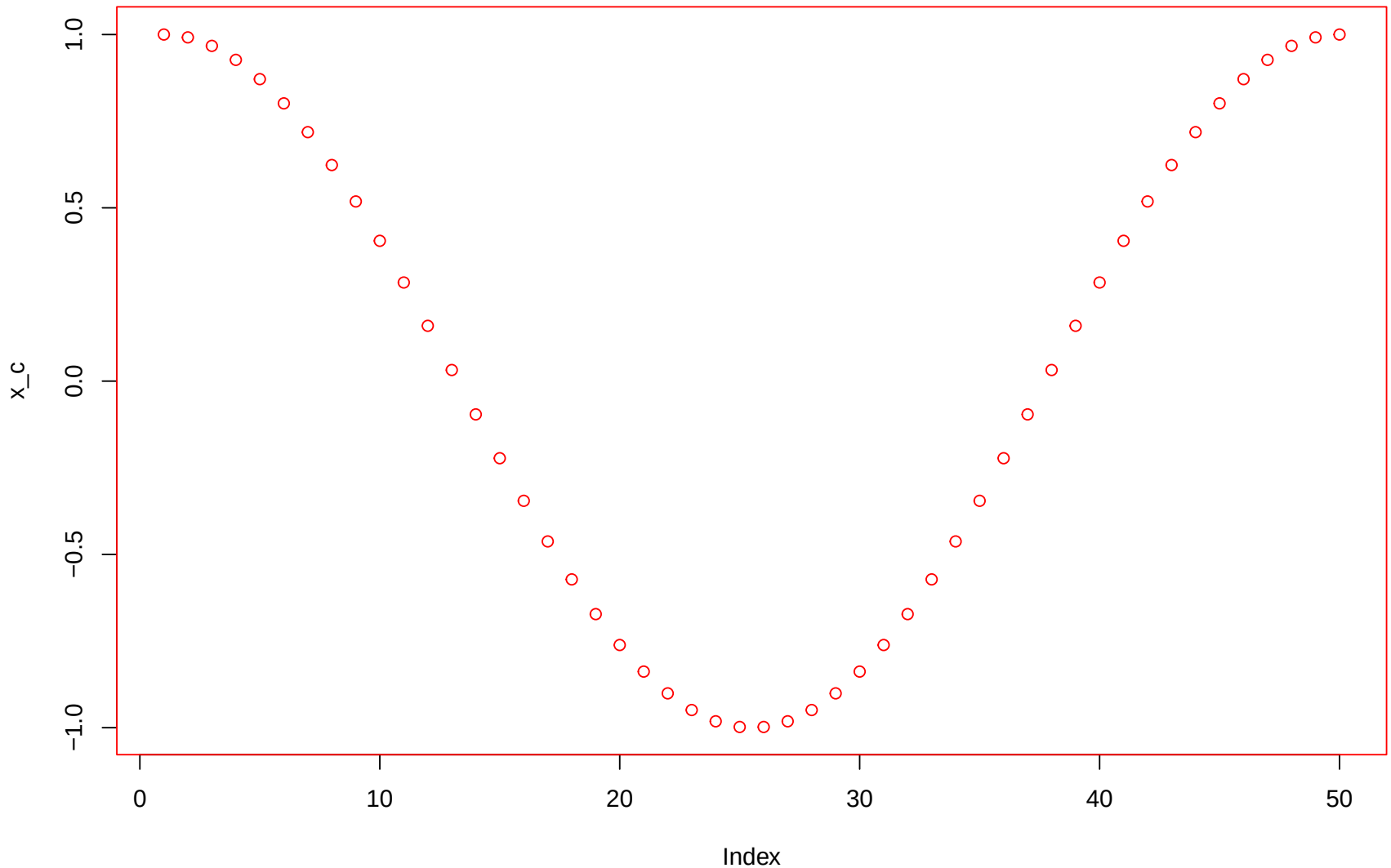
```
[1] "black"
```

You can modify it by

```
par(col = "red")
```

From now on all your plots will be red!

```
plot(x_c)
```



Generally, you don't want this!

A useful global parameter is:

```
par()$mfrow
```

```
[1] 1 1
```

Tells you how plot will be displayed on its own page.

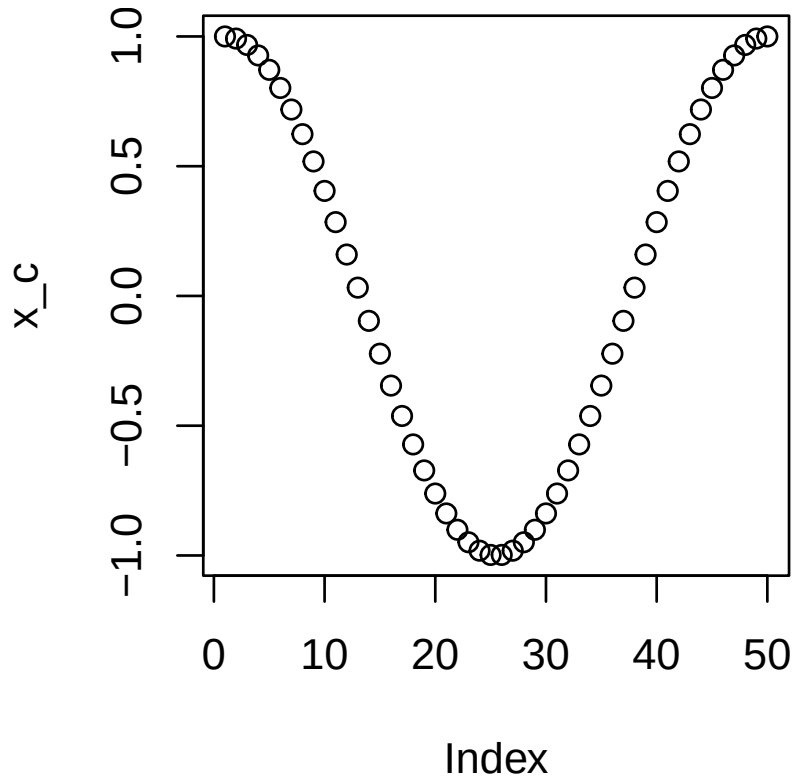
You can change that by doing:

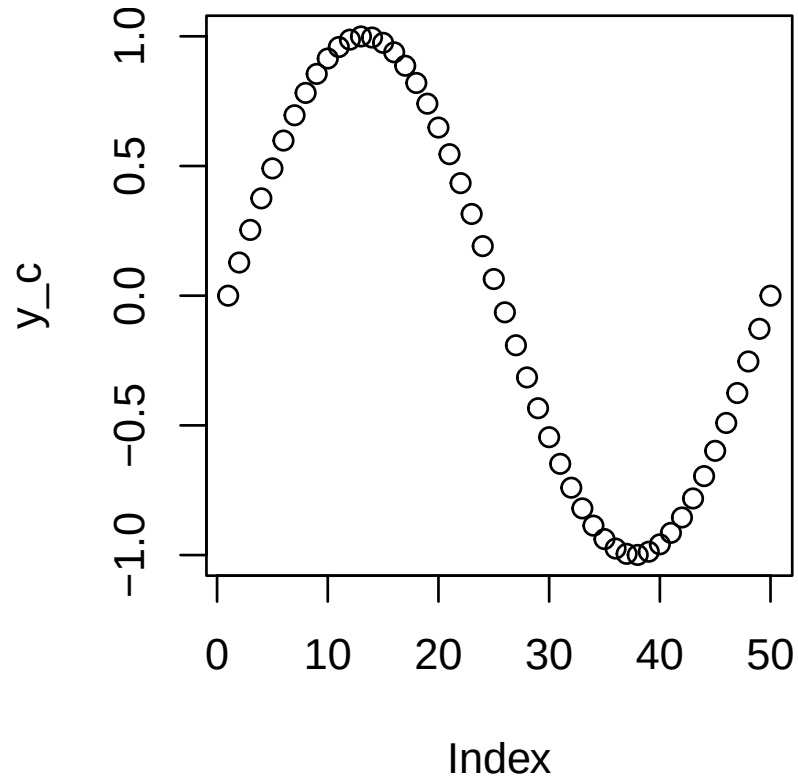
```
par(mfrow = c(1, 2))
```

So when you do

```
plot(x_c)
plot(y_c)
```

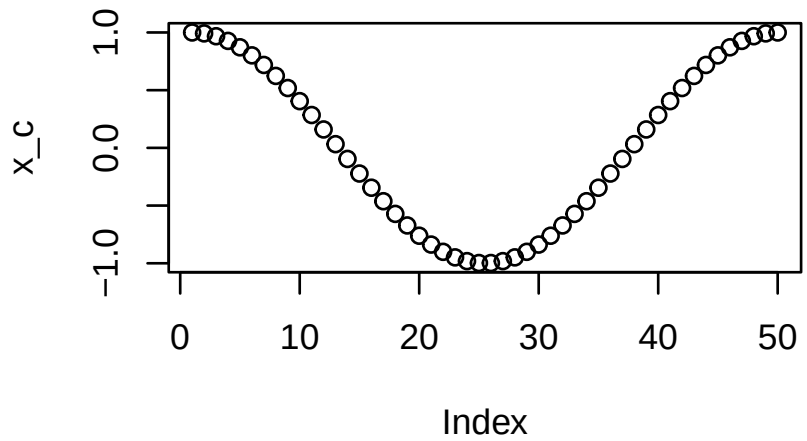
You get:

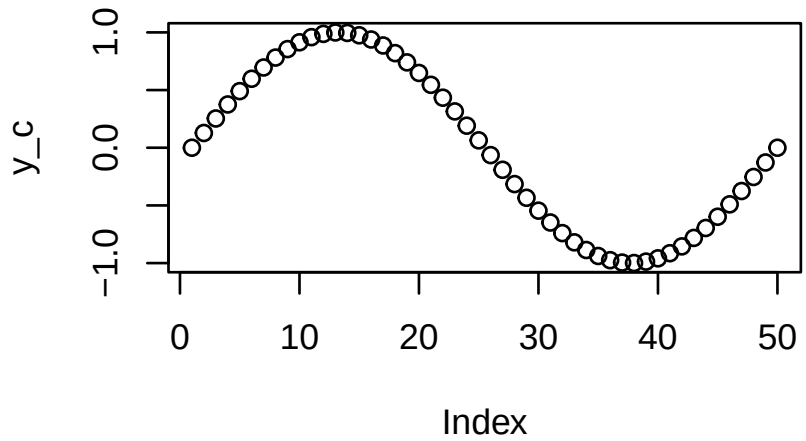

x_c
1.0
0.5
0.0
−0.5
−1.0
0
10
20
30
40
50
Index

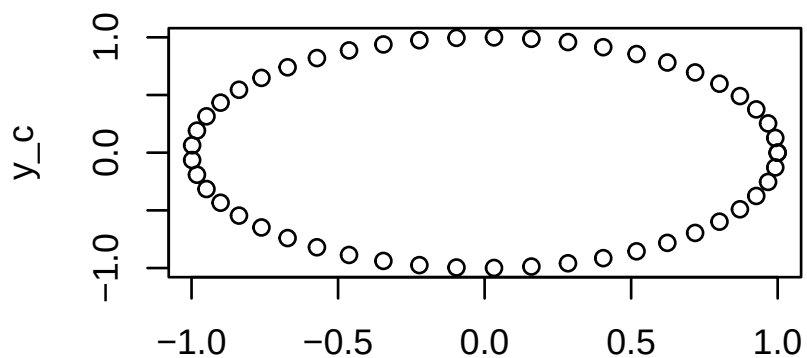

y_c
1.0
0.5
0.0
−0.5
−1.0
0
10
20
30
40
50
Index

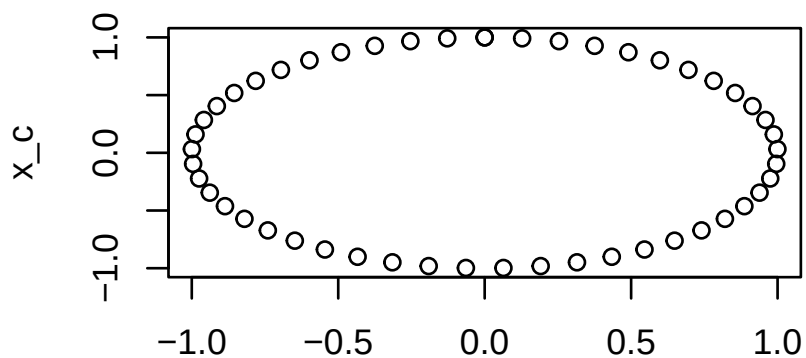
Or

```
par(mfrow = c(2, 2))
plot(x_c); plot(y_c)
plot(x_c, y_c); plot(y_c, x_c)
```





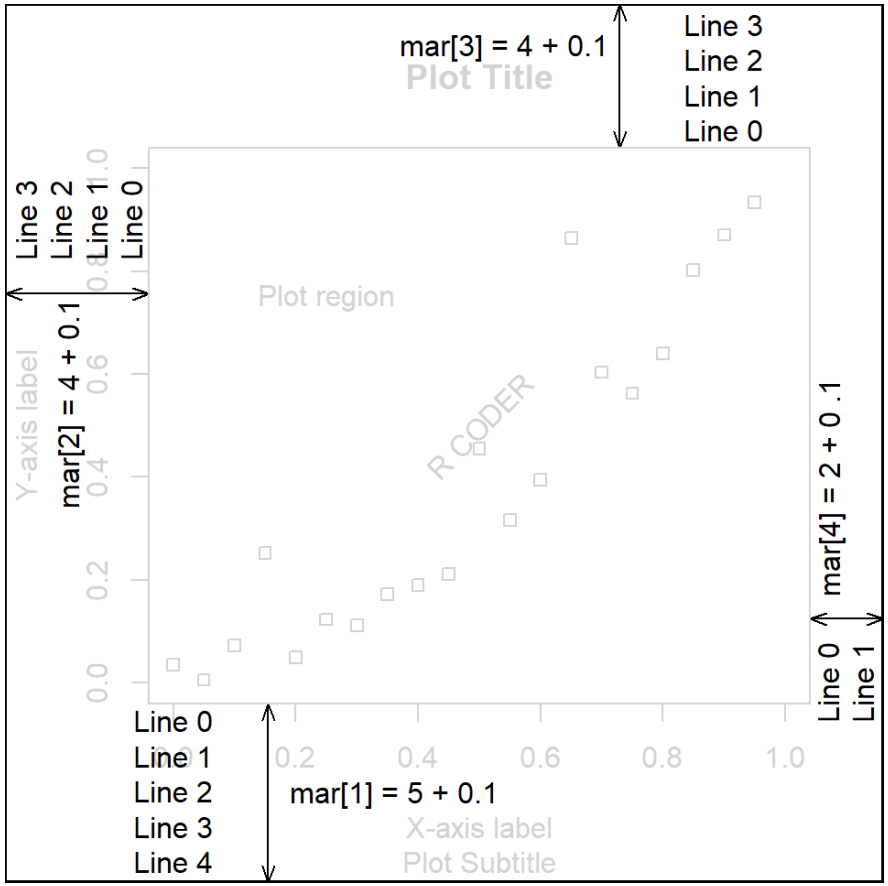




Another useful graphical parameter is:

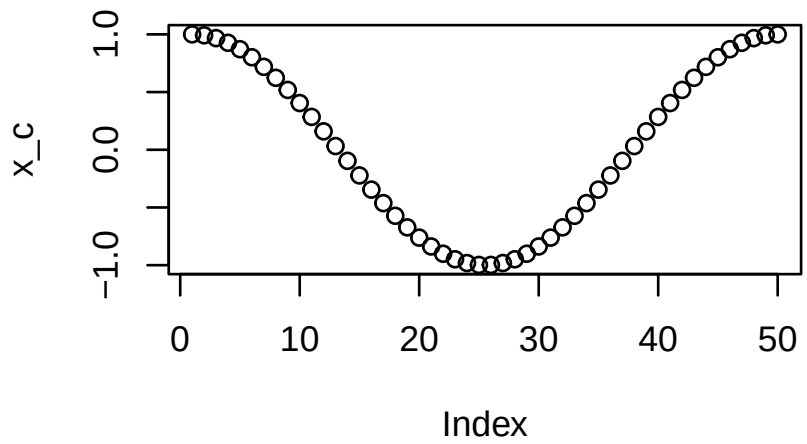
```
par()$mar
```

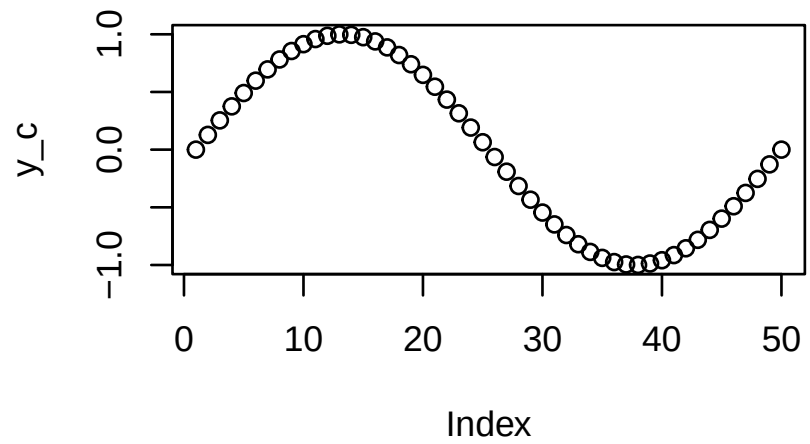
```
[1] 5.1 4.1 4.1 2.1
```

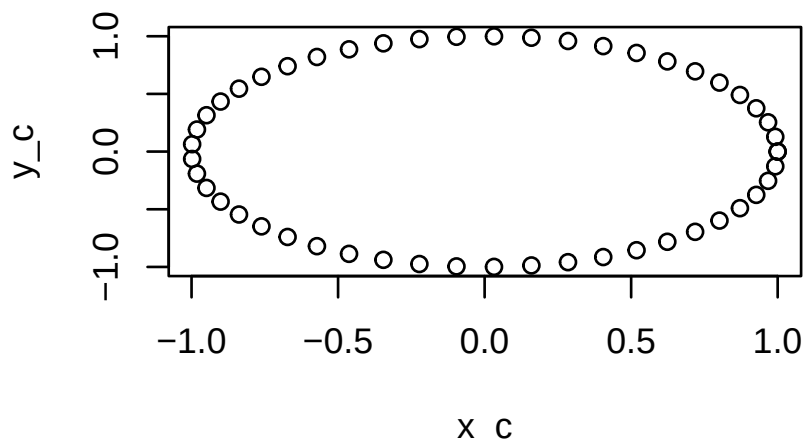


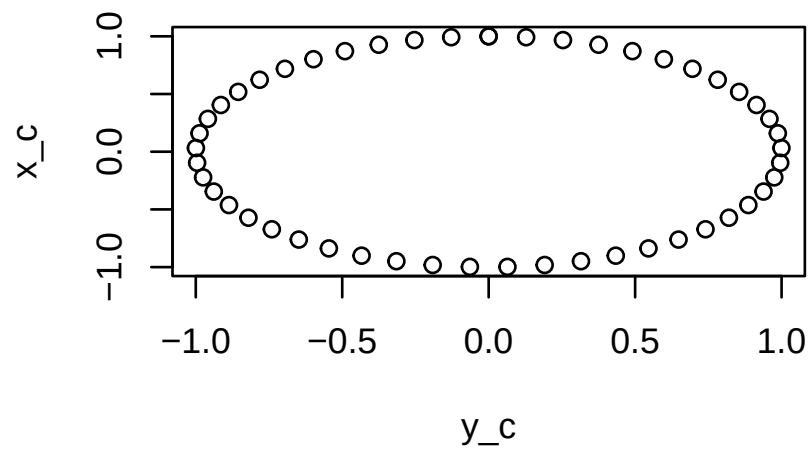
From https://r-charts.com/base-r/margins

```r
par(mfrow = c(2, 2))
plot(x_c); plot(y_c)
plot(x_c, y_c); plot(y_c, x_c)
```

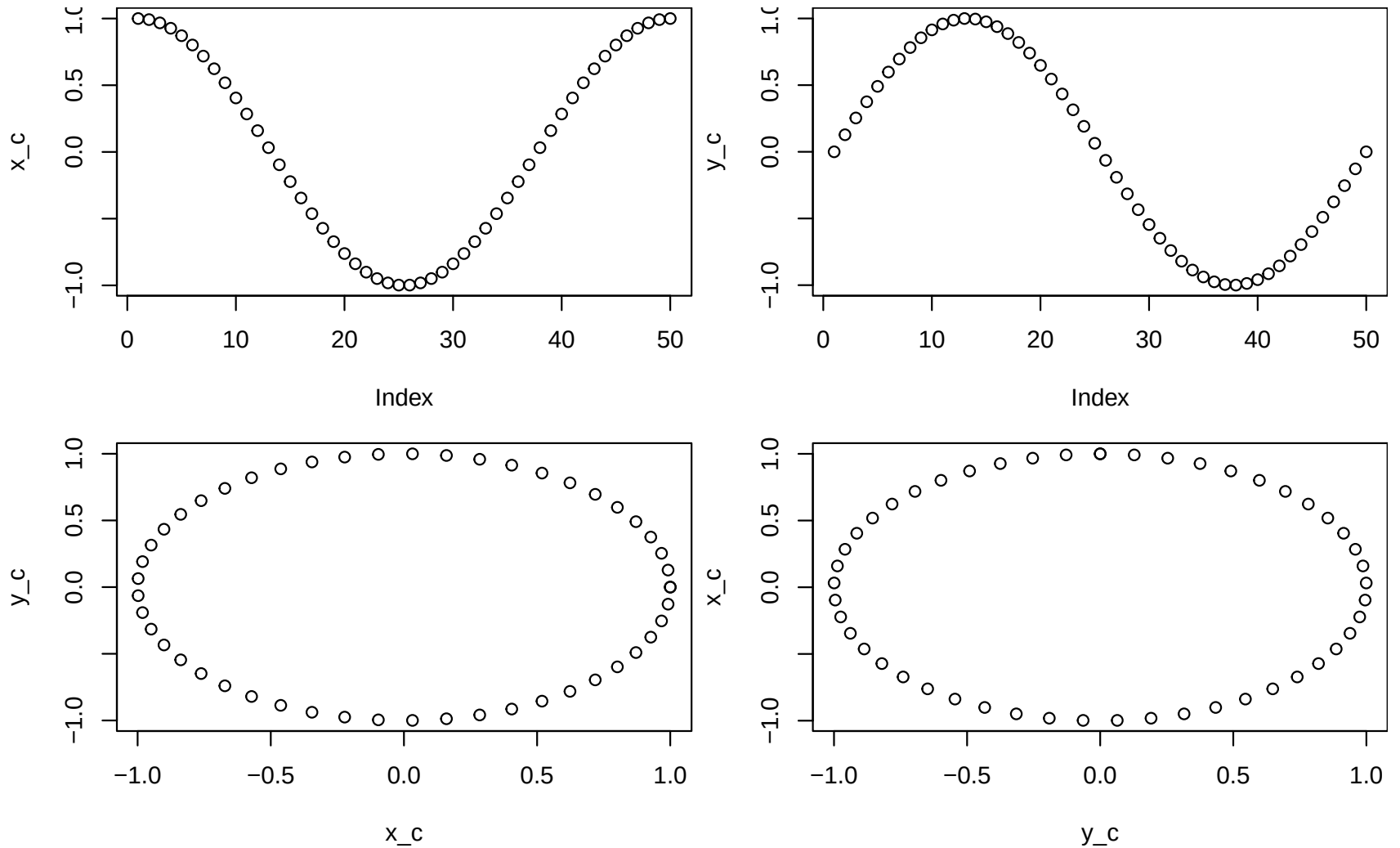








```r
par(mfrow = c(2, 2), mar = c(5.1, 4.2, 0, 0))
plot(x_c); plot(y_c)
plot(x_c, y_c); plot(y_c, x_c)
```



Summary on plotting in R

The main function we will use is `plot(x, y, ...)`.

It creates a plot, its limits can be adjusted with `xlim` and `ylim`.

Title and axes labels set using `xlab`, `ylab`, `main`.

Layers can be added using `points()`, `lines()`, `text()`...

Aesthetics can be adjusted with `col`, `lwd`, `pch`, ...

Layout and margins can be changed using

```
par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
```

Plotting functions are called for their side effects.

Homework

This exercise is inspired by a scene in the TV show "The Office":

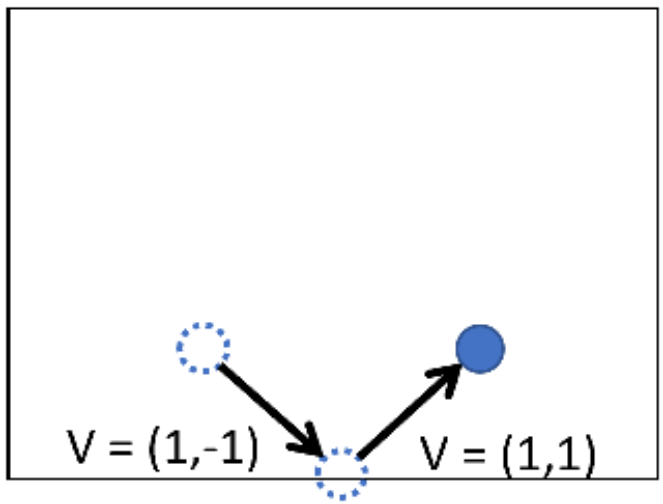
https://www.youtube.com/watch?v=QOtuX0jL85Y

We are going to write an animated visualization that mimics the DVD logo bouncing around the edge of the TV.

1. Create an empty plot, whose x-axis and y-axis range from -5 to 5.
2. Initialize two vectors:
   - $x = (0, 0)$ and $v = (0.23456, 0.12345)$.
   - Draw the vector x as a **red** dot in the plot.
   - Add plot title: "point simulation, press ESC to stop."
     - Hint: `?title`

3. Now let us write a function that updates the point's `x` location based on the velocity vector `v`.

   - If the position of `x` is slightly out of the box, the point will be **bounced back**.



3. Write a function `update(x,v)` that takes two vectors `x` and `v` as inputs and returns the updated `x` and `v`.
   - First, the function checks if `x` is outside of the boundary.
   - If so, it updates the velocity vector by "reflecting it". See the picture in the previous slide.
   - Then, update `x <- x + v`
   - Finally, `return( list(x,v) )`
   - The `list` combines two vectors and allows you to return multiple values from a function.
   - Hint: `x[2] > 5 || x[2] < -5` is true if `x` is above the ceiling or is below the floor.

Note on lists, if you do `z <- list(x,v)` than `z[[1]]` returns `x` and `z[[2]]` returns `v`.

That's all you need to know, but if you prefer you can use a matrix to store `x` and `v`.

4. Your code should look like this:

```
update <- function(x,v){
  # TODO: Check the boundary collision,
  #       and update v and x.
  return(list(x,v))
}

x <- c(0,0) #the initial position of x
v <- c(0.23456,0.12345) #the initial velocity of x

while(T){
  # TODO: create a new plot and draw x's current
  #       location
  x <- xv[[1]]
  v <- xv[[2]]
  Sys.sleep(0.1)  # wait a bit to give Rstudio
                  # the time to plot
}
```

Your plot should look like this:

https://github.com/anewgithubname/MATH10017-2022/blob/main/lecs/lab14_1.mp4

In the TV show, each time the logo bounces, it changes color randomly. Could you revise your code so that the point in your plot also changes its color every time it bounces?